

Titanic Frontiers

September 25, 2026

1 Default Data Pipeline

```
[1]: %matplotlib inline
# Pandas is a nice utility that enables some easy data manipulation,
# especially from a csv
import pandas as pd
# Numpy lets us work with arrays
import numpy as np
import random
# Sklearn provides various modules with a common API
from sklearn import svm, tree, neighbors, neural_network, ensemble
from sklearn.model_selection import train_test_split
# For optimizing our models' hyperparameters
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from matplotlib import pyplot as plt

# Set random seed
random.seed(100)
np.random.seed(100)
```

```
[2]: # Read in our csv files downloaded from Kaggle
train_data = pd.read_csv('train.csv')
test_data = pd.read_csv('test.csv')
```

```
[3]: random.seed(100)
np.random.seed(100)

train_data.drop(columns=['Name', 'Ticket', 'Cabin'], inplace=True)
train_data.set_index(keys=['PassengerId'], drop=True, inplace=True)

test_data.drop(columns=['Name', 'Ticket', 'Cabin'], inplace=True)
test_data.set_index(keys=['PassengerId'], drop=True, inplace=True)
```

```

train_nan_map = {'Age': train_data['Age'].mean(), 'Fare': train_data['Fare'].
    ↪mean(), 'Embarked': train_data['Embarked'].mode()[0]}
test_nan_map = {'Age': test_data['Age'].mean(), 'Fare': test_data['Fare'].
    ↪mean(), 'Embarked': test_data['Embarked'].mode()[0]}

train_data.fillna(value=train_nan_map, inplace=True)
test_data.fillna(value=test_nan_map, inplace=True)

columns_map = {'Embarked': {'C': 0, 'Q': 1, 'S': 2}, 'Sex': {'male': 0,
    ↪'female': 1}}
# Adding below line to surpress deprication warnings
pd.set_option('future.no_silent_downcasting', True)
train_data.replace(columns_map, inplace=True)
test_data.replace(columns_map, inplace=True)

X_train = train_data.loc[:, train_data.columns != 'Survived']
y_train = train_data.loc[:, 'Survived']

X_train, X_test, y_train, y_test = train_test_split(X_train, y_train,
    ↪test_size=0.33, random_state=100)

```

```
[4]: y_truth = y_test.values
```

```

[5]: # Instantiate Containers and Update Function
x_axis1 = [] # list of lists
y_axis1 = [] # list of lists
FPR_axis = []
FNR_axis = []
name_labels = ['Boost', 'SVC', 'LR', 'KNN', 'RF']

def ordered_pair(tn, fp, fn, tp):
    # x-axis is the False Positive Rate, y-axis is the False Negative Rate
    fpr = fp / (fp + tn)
    fnr = fn / (fn + tp)
    return (fpr, fnr)

```

2 Milestone 1: default hyperparameters, default data pipeline

```

[6]: # Andy - Boosted Tree
boost_clf = ensemble.GradientBoostingClassifier(random_state=100)
# Train the model
boost_clf.fit(X_train, y_train)

# Make predictions
y_pred = boost_clf.predict(X_test)

```

```

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 154
 False Positives: 18
 False Negatives: 39
 True Positives: 84
 FPR = 0.1047 FNR = 0.3171

```

[7]: # Stevie - SVC
svc_clf = svm.SVC()

# Train the model
svc_clf.fit(X_train.values, y_train.values)

# Make predictions
y_pred = svc_clf.predict(X_test.values)

tn, fp, fn, tp = confusion_matrix(y_test.values, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 162
 False Positives: 10
 False Negatives: 94
 True Positives: 29
 FPR = 0.0581 FNR = 0.7642

```

[8]: # Asher - Logistic Classification
LC = LogisticRegression()

# Train the model

```

```

LC.fit(X_train, y_train)

# Make predictions
y_pred = LC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test.values, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 152
 False Positives: 20
 False Negatives: 42
 True Positives: 81
 FPR = 0.1163 FNR = 0.3415

```

[9]: # Shrey - KNN
KNNC = neighbors.KNeighborsClassifier()

KNNC.fit(X_train, y_train)
y_pred = KNNC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test.values, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 137
 False Positives: 35
 False Negatives: 53
 True Positives: 70
 FPR = 0.2035 FNR = 0.4309

```

[10]: # Val - Random Forest
RFC = ensemble.RandomForestClassifier()

```

```

# Train the model
RFC.fit(X_train, y_train)

# Make predictions
y_pred = RFC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test.values, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

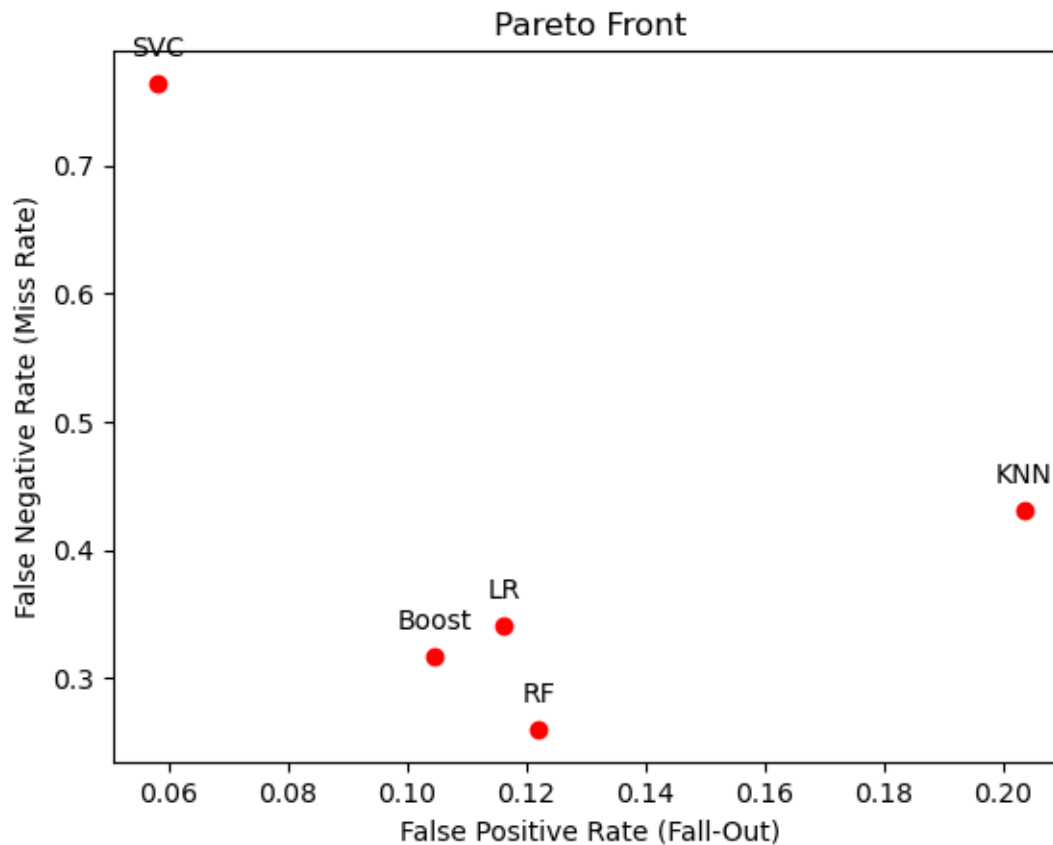
True Negatives: 151
False Positives: 21
False Negatives: 32
True Positives: 91
FPR = 0.1221 FNR = 0.2602

```

```

[11]: # Plot output
plt.scatter(FPR_axis, FNR_axis, color='r')
for i, label in enumerate(name_labels):
    # xy=(x, y) is the point position
    # xytext=(offset_x, offset_y) nudges the text so it doesn't overlap the dot
    plt.annotate(
        label,
        (FPR_axis[i], FNR_axis[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center'
    )
plt.xlabel("False Positive Rate (Fall-Out)")
plt.ylabel("False Negative Rate (Miss Rate)")
plt.title("Pareto Front")
plt.show()

```



3 Milestone 2: default hyperparameters, new data pipeline

```
[12]: scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

[13]: # Clear Containers and Build Final x and y axes
x_axis1.append(FPR_axis)
y_axis1.append(FNR_axis)

FPR_axis = []
FNR_axis = []

[14]: # Andy - Boosted Tree
boost_clf = ensemble.GradientBoostingClassifier(random_state=100)
# Train the model
boost_clf.fit(X_train, y_train)
```

```

# Make predictions
y_pred = boost_clf.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 154
 False Positives: 18
 False Negatives: 39
 True Positives: 84
 FPR = 0.1047 FNR = 0.3171

```

[15]: # Stevie - SVC
svc_clf = svm.SVC()

# Train the model
svc_clf.fit(X_train, y_train)

# Make predictions
y_pred = svc_clf.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

True Negatives: 157
 False Positives: 15
 False Negatives: 42
 True Positives: 81
 FPR = 0.0872 FNR = 0.3415

```
[16]: # Asher - Logistic Classification
LC = LogisticRegression()

# Train the model
LC.fit(X_train, y_train)

# Make predictions
y_pred = LC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')
```

True Negatives: 151
False Positives: 21
False Negatives: 42
True Positives: 81
FPR = 0.1221 FNR = 0.3415

```
[17]: # Shrey - KNN
KNNC = neighbors.KNeighborsClassifier()

KNNC.fit(X_train, y_train)
y_pred = KNNC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')
```

True Negatives: 153
False Positives: 19
False Negatives: 36
True Positives: 87

FPR = 0.1105 FNR = 0.2927

```
[18]: # Val - Random Forest
RFC = ensemble.RandomForestClassifier()

# Train the model
RFC.fit(X_train, y_train)

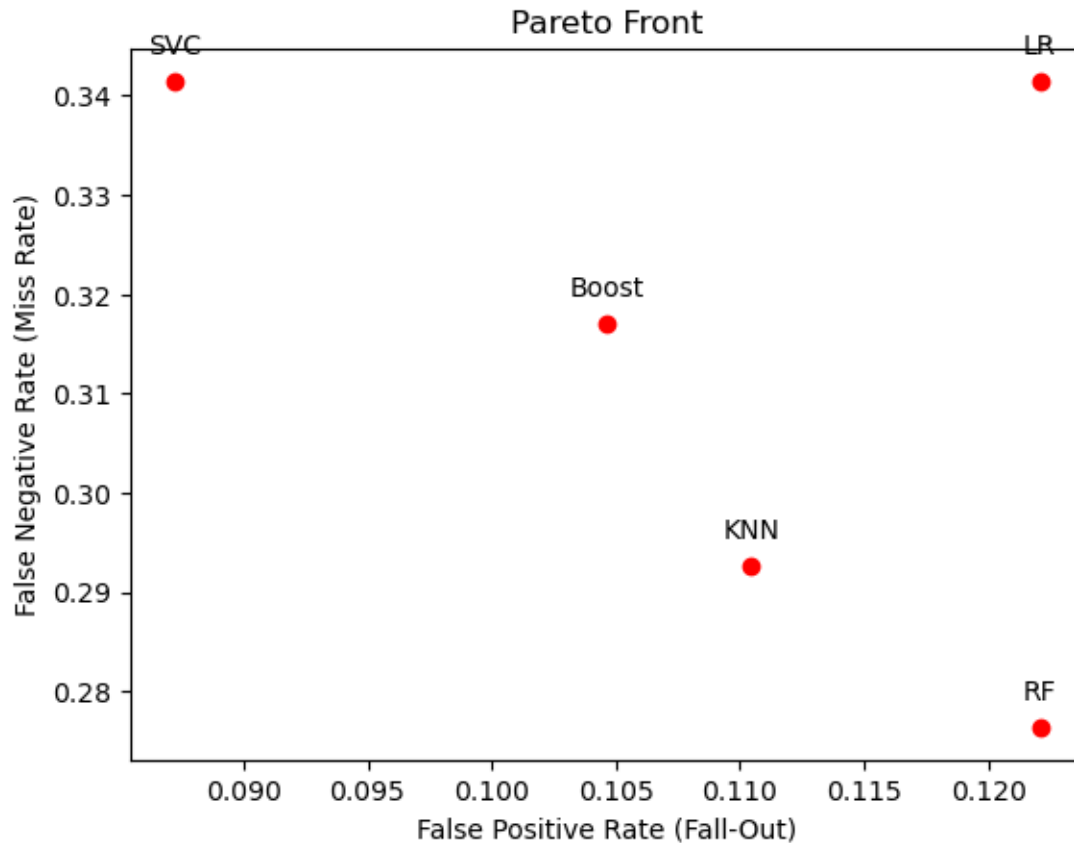
# Make predictions
y_pred = RFC.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')
```

True Negatives: 151
False Positives: 21
False Negatives: 34
True Positives: 89
FPR = 0.1221 FNR = 0.2764

```
[19]: # Plot output
plt.scatter(FPR_axis, FNR_axis, color='r')
for i, label in enumerate(name_labels):
    # xy=(x, y) is the point position
    # xytext=(offset_x, offset_y) nudges the text so it doesn't overlap the dot
    plt.annotate(
        label,
        (FPR_axis[i], FNR_axis[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center'
    )
plt.xlabel("False Positive Rate (Fall-Out)")
plt.ylabel("False Negative Rate (Miss Rate)")
plt.title("Pareto Front")
plt.show()
```



4 Milestone 3: Initial Grid Search, New Data Pipeline

```
[57]: x_axis1.append(FPR_axis)
      y_axis1.append(FNR_axis)
```

```
FPR_axis = []
FNR_axis = []
```

```
[58]: # Andy - Boosted Tree
```

```
param_grid = {
    'learning_rate': [.01, .02, .04],
    'n_estimators': [50, 60, 70, 80, 90],
    'max_depth': [2,3,4],
    'subsample': [0.9, 1.0, 1.1]
}
```

```
boost_clf = ensemble.GradientBoostingClassifier(random_state=100)
```

```

grid_search = GridSearchCV(
    estimator=boost_clf,
    param_grid=param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

optimal_boost_clf = grid_search.best_estimator_
print("Best C:", grid_search.best_params_)
print("Best CV ROC AUC:", grid_search.best_score_)
print("Held-out test accuracy:", optimal_boost_clf.score(X_test, y_test))

y_pred = optimal_boost_clf.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

Fitting 5 folds for each of 135 candidates, totalling 675 fits
 Best C: {'learning_rate': 0.04, 'max_depth': 3, 'n_estimators': 50, 'subsample': 1.0}
 Best CV ROC AUC: 0.8822952227291271
 Held-out test accuracy: 0.8
 True Negatives: 154
 False Positives: 18
 False Negatives: 41
 True Positives: 82
 FPR = 0.1047 FNR = 0.3333
 C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection_validation.py:528: FitFailedWarning:
 225 fits failed out of a total of 675.
 The score on these train-test partitions for these parameters will be set to

nan.

If these failures are not expected, you can try to debug them by setting `error_score='raise'`.

Below are more details about the failures:

225 fits failed with the following error:

Traceback (most recent call last):

```
File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
    ~~~~~^~~~~~
```

```
File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\base.py", line 1382, in wrapper
    estimator._validate_params()
    ~~~~~^
```

```
File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\base.py", line 436, in _validate_params
    validate_parameter_constraints(
    ~~~~~^
```

```
        self._parameter_constraints,
        ~~~~~
        self.get_params(deep=False),
        ~~~~~
        caller_name=self.__class__.__name__,
        ~~~~~
    )
    ^
```

```
File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\utils\_param_validation.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
    ...<2 lines>...
    )
```

sklearn.utils._param_validation.InvalidParameterError: The 'subsample' parameter of GradientBoostingClassifier must be a float in the range (0.0, 1.0]. Got 1.1 instead.

```
warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection\_search.py:1108: UserWarning: One or more of the test scores are non-finite: [0.8545505  0.85025422      nan 0.85594723
0.84979196      nan
0.8553954  0.84979196      nan 0.85631617 0.84885815      nan
0.85916901 0.85120038      nan 0.87181167 0.8646783      nan
0.87090322 0.86723417      nan 0.87132816 0.86719997      nan
0.87255922 0.86747181      nan 0.87284783 0.86883367      nan
0.86628017 0.86449225      nan 0.86735763 0.8630313      nan
```

```

0.8700525  0.86742331      nan 0.8706983  0.86787367      nan
0.87467747 0.86896515      nan 0.85689985 0.85216848      nan
0.85695521 0.85327126      nan 0.86208569 0.85733068      nan
0.86135668 0.85711585      nan 0.86209541 0.86110992      nan
0.87158503 0.86981853      nan 0.87721835 0.87275086      nan
0.88037573 0.8766157       nan 0.88076092 0.87733083      nan
0.88064143 0.8811111       nan 0.87182828 0.87035142      nan
0.87331569 0.87125909      nan 0.87389785 0.87375756      nan
0.87515296 0.87391206      nan 0.87796984 0.87445412      nan
0.86477364 0.8610792       nan 0.86535785 0.86463817      nan
0.86573109 0.86585609      nan 0.86900089 0.86452053      nan
0.866872   0.8648636       nan 0.87738771 0.88229522      nan
0.87644908 0.87950524      nan 0.87643934 0.87678564      nan
0.87623031 0.87812364      nan 0.87520181 0.87694623      nan
0.87582753 0.87683439      nan 0.87799993 0.87773538      nan
0.87752652 0.87554071      nan 0.87779133 0.8762974       nan
0.87638999 0.87589317      nan]
warnings.warn(

```

```

[59]: # Stevie - SVC

C_vals = [0.01, 0.1, 1, 10]
cw_vals = [None, 'balanced']
gamma_vals = ['scale', 'auto', 0.01, 0.1, 1]
degree_vals = [2, 3, 4]

param_grid = [
    # RBF - the default, usually the best starting point
    {'kernel': ['rbf'], 'C': C_vals, 'class_weight': cw_vals, 'gamma': ↵
    ↵gamma_vals},
    # Linear - no gamma, no degree
    {'kernel': ['linear'], 'C': C_vals, 'class_weight': cw_vals},
    # Polynomial - has degree AND gamma
    {'kernel': ['poly'], 'C': C_vals, 'class_weight': cw_vals, 'gamma': ↵
    ↵gamma_vals, 'degree': degree_vals},
    # Sigmoid - has gamma
    {'kernel': ['sigmoid'], 'C': C_vals, 'class_weight': cw_vals, 'gamma': ↵
    ↵gamma_vals},
]

svc_grid = GridSearchCV(
    svm.SVC(probability=False),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
)

```

```

svc_grid.fit(X_train, y_train)

best_svc = svc_grid.best_estimator_
print("Best C:", svc_grid.best_params_)
print("Best CV ROC AUC:", svc_grid.best_score_)
print("Held-out test accuracy:", best_svc.score(X_test, y_test))

y_pred = best_svc.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'C': 1, 'class_weight': 'balanced', 'gamma': 0.01, 'kernel': 'rbf'}
Best CV ROC AUC: 0.8583775638885799
Held-out test accuracy: 0.7762711864406779
True Negatives: 148
False Positives: 24
False Negatives: 42
True Positives: 81
FPR = 0.1395 FNR = 0.3415

```

```

[60]: # Asher - Logistic Classification
param_grid = {
    "C": [0.01, 0.1, 1, 10, 100],
    "penalty": ["l1", "l2"],
    "solver": ["liblinear"], # liblinear supports both l1 and l2
    "class_weight": [None, "balanced"],
}
grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5,
    scoring="roc_auc")
grid.fit(X_train, y_train)

log_reg = grid.best_estimator_

```

```

y_pred = log_reg.predict(X_test)
y_truth = y_test

print("Best C:", grid.best_params_)
print("Best CV ROC AUC:", grid.best_score_)
print("Held-out test accuracy:", log_reg.score(X_test, y_test))

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'C': 0.1, 'class_weight': 'balanced', 'penalty': 'l2', 'solver':
'liblinear'}
Best CV ROC AUC: 0.8531126256444495
Held-out test accuracy: 0.7762711864406779
True Negatives: 143
False Positives: 29
False Negatives: 37
True Positives: 86
FPR = 0.1686 FNR = 0.3008

```

```

[61]: # Shrey - KNN
param_grid = {
    'n_neighbors': [7, 9, 11, 13, 15, 17, 19, 21, 23, 25],
    'weights': ['uniform', 'distance'],
    'p': [1, 2]
}

knn_grid = GridSearchCV(
    neighbors.KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='roc_auc'
)

knn_grid.fit(X_train, y_train)    # already scaled, already DataFrame

best_knn = knn_grid.best_estimator_

```

```

print("Best C:", knn_grid.best_params_)
print("Best CV ROC AUC:", knn_grid.best_score_)
print("Held-out test accuracy:", knn_grid.score(X_test, y_test))

y_pred = best_knn.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'n_neighbors': 13, 'p': 1, 'weights': 'uniform'}
Best CV ROC AUC: 0.8605189903935313
Held-out test accuracy: 0.8536112686708263
True Negatives: 161
False Positives: 11
False Negatives: 41
True Positives: 82
FPR = 0.0640 FNR = 0.3333

```

[62]: *# Val - Random Forest*

```

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [None, 'balanced'],
}

RF_grid = GridSearchCV(
    ensemble.RandomForestClassifier(random_state=100),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
)

```

```

RF_grid.fit(X_train, y_train)

best_rf = RF_grid.best_estimator_

print("Best C:", RF_grid.best_params_)
print("Best CV ROC AUC:", RF_grid.best_score_)
print("Held-out test accuracy:", RF_grid.score(X_test, y_test))

y_pred = best_rf.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'class_weight': 'balanced', 'max_depth': None, 'min_samples_leaf': 4,
'min_samples_split': 10, 'n_estimators': 300}
Best CV ROC AUC: 0.8748749304551018
Held-out test accuracy: 0.8647428625449045
True Negatives: 150
False Positives: 22
False Negatives: 30
True Positives: 93
FPR = 0.1279 FNR = 0.2439

```

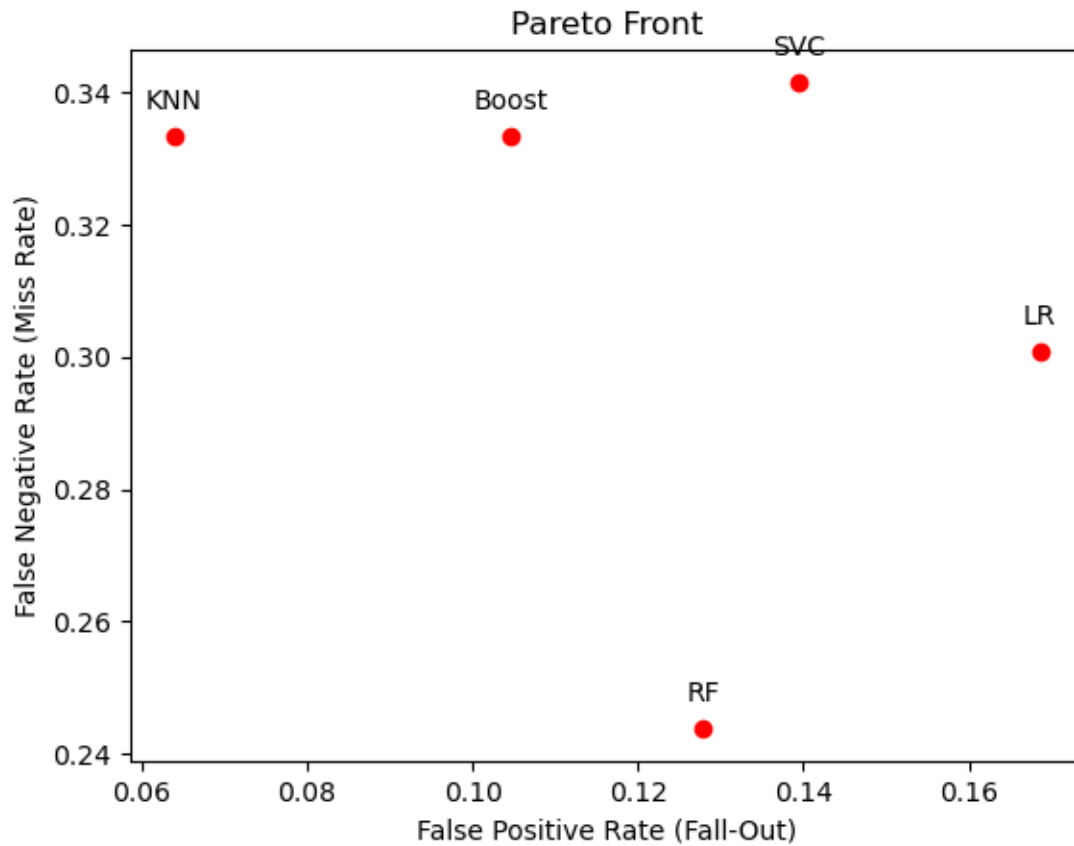
```

[63]: # Plot output
plt.scatter(FPR_axis, FNR_axis, color='r')

for i, label in enumerate(name_labels):
    # xy=(x, y) is the point position
    # xytext=(offset_x, offset_y) nudges the text so it doesn't overlap the dot
    plt.annotate(
        label,
        (FPR_axis[i], FNR_axis[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center'
    )

```

```
plt.xlabel("False Positive Rate (Fall-Out)")
plt.ylabel("False Negative Rate (Miss Rate)")
plt.title("Pareto Front")
plt.show()
```



5 Milestone 4: Final Grid Search, New Data Pipeline

```
[87]: x_axis1.append(FPR_axis)
      y_axis1.append(FNR_axis)
```

```
FPR_axis = []
FNR_axis = []
```

```
[88]: # Andy - Boosted Tree
```

```
param_grid = {
    'learning_rate': [.03, .04, .05],
    'n_estimators': [45, 48, 50, 52, 55],
```

```

        'max_depth': [2,3,4],
        'subsample': [0.9, 1.0, 1.1]
    }

    boost_clf = ensemble.GradientBoostingClassifier(random_state=100)

    grid_search = GridSearchCV(
        estimator=boost_clf,
        param_grid=param_grid,
        cv=5,
        scoring='roc_auc',
        n_jobs=-1,
        verbose=1
    )

    grid_search.fit(X_train, y_train)

    optimal_boost_clf = grid_search.best_estimator_
    print("Best C:", grid_search.best_params_)
    print("Best CV ROC AUC:", grid_search.best_score_)
    print("Held-out test accuracy:", optimal_boost_clf.score(X_test, y_test))

    y_pred = optimal_boost_clf.predict(X_test)
    y_truth = y_test

    tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

    print("True Negatives:", tn)
    print("False Positives:", fp)
    print("False Negatives:", fn)
    print("True Positives:", tp)

    output = ordered_pair(tn, fp, fn, tp)
    FPR_axis.append(output[0])
    FNR_axis.append(output[1])
    print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

Fitting 5 folds for each of 135 candidates, totalling 675 fits
 Best C: {'learning_rate': 0.04, 'max_depth': 3, 'n_estimators': 52, 'subsample': 1.0}
 Best CV ROC AUC: 0.8827627220800414
 Held-out test accuracy: 0.8
 True Negatives: 154
 False Positives: 18
 False Negatives: 41
 True Positives: 82

FPR = 0.1047 FNR = 0.3333

C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection_validation.py:528: FitFailedWarning:
225 fits failed out of a total of 675.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:

```
-----  
225 fits failed with the following error:  
Traceback (most recent call last):  
  File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 866, in _fit_and_score  
    estimator.fit(X_train, y_train, **fit_params)  
    ~~~~~~  
  File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\base.py", line 1382, in wrapper  
    estimator._validate_params()  
    ~~~~~~  
  File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\base.py", line 436, in _validate_params  
    validate_parameter_constraints(  
    ~~~~~~  
        self._parameter_constraints,  
        ~~~~~~  
        self.get_params(deep=False),  
        ~~~~~~  
        caller_name=self.__class__.__name__,  
        ~~~~~~  
    )  
    ^  
  File "C:\Users\thund\anaconda3\Lib\site-packages\sklearn\utils\_param_validation.py", line 98, in validate_parameter_constraints  
    raise InvalidParameterError(  
    ...<2 lines>...  
    )  
sklearn.utils._param_validation.InvalidParameterError: The 'subsample' parameter of GradientBoostingClassifier must be a float in the range (0.0, 1.0]. Got 1.1 instead.
```

```
warnings.warn(some_fits_failed_message, FitFailedWarning)  
C:\Users\thund\anaconda3\Lib\site-packages\sklearn\model_selection\_search.py:1108: UserWarning: One or more of the test scores are non-finite: [0.85745143 0.85714886 nan 0.85644593
```

```

0.85769113      nan
0.85683828 0.85711329      nan 0.85735322 0.8560182      nan
0.85662941 0.85741948      nan 0.87409987 0.8758338      nan
0.87638834 0.8765569      nan 0.8767457 0.8765854      nan
0.87890352 0.87711672      nan 0.87901276 0.87723673      nan
0.87152984 0.87400115      nan 0.87119471 0.87430618      nan
0.87141397 0.87412574      nan 0.87092775 0.87433626      nan
0.87270769 0.87433369      nan 0.86317573 0.86139036      nan
0.86438764 0.86153513      nan 0.86477364 0.8610792      nan
0.86456381 0.86068845      nan 0.86495535 0.86220796      nan
0.87707025 0.88057659      nan 0.87919006 0.88184923      nan
0.87738771 0.88229522      nan 0.87750694 0.88276272      nan
0.87629192 0.88167838      nan 0.87458163 0.87640041      nan
0.87694545 0.87580112      nan 0.87582753 0.87683439      nan
0.87619623 0.877014      nan 0.87728731 0.87800504      nan
0.86068294 0.86456601      nan 0.86210503 0.86316088      nan
0.86343181 0.86292224      nan 0.86394935 0.86461901      nan
0.86591796 0.86501419      nan 0.87571645 0.88107551      nan
0.87689955 0.8802639      nan 0.87699563 0.87920809      nan
0.87771732 0.87853766      nan 0.87731585 0.87570231      nan
0.87452505 0.87693288      nan 0.87709777 0.87814983      nan
0.87576697 0.87675749      nan 0.87719447 0.87711358      nan
0.87658125 0.87725425      nan]
warnings.warn(

```

```

[89]: # Stevie - SVC

C_vals = [0.5, 1, 1.5, 2]
cw_vals = ['balanced']
gamma_vals = [.005, 0.01, .02, .03]
degree_vals = [2, 3, 4]

param_grid = [
    # RBF - the default, usually the best starting point
    {'kernel': ['rbf'], 'C': C_vals, 'class_weight': cw_vals, 'gamma': ↵
    ↵gamma_vals},
]

svc_grid = GridSearchCV(
    svm.SVC(probability=False),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
)

svc_grid.fit(X_train, y_train)

```

```

best_svc = svc_grid.best_estimator_
print("Best C:", svc_grid.best_params_)
print("Best CV ROC AUC:", svc_grid.best_score_)
print("Held-out test accuracy:", best_svc.score(X_test, y_test))

y_pred = best_svc.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'C': 2, 'class_weight': 'balanced', 'gamma': 0.005, 'kernel': 'rbf'}
Best CV ROC AUC: 0.8615053410481807
Held-out test accuracy: 0.7762711864406779
True Negatives: 148
False Positives: 24
False Negatives: 42
True Positives: 81
FPR = 0.1395 FNR = 0.3415

```

```

[90]: # Asher - Logistic Classification
param_grid = {
    "C": [.05, .075, .1, .2, .3],
    "penalty": ["l1", "l2"],
    "solver": ["liblinear"], # liblinear supports both l1 and l2
    "class_weight": ['balanced', None],
}
grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5,
    ↪scoring="roc_auc")
grid.fit(X_train, y_train)

log_reg = grid.best_estimator_

y_pred = log_reg.predict(X_test)
y_truth = y_test

```

```

print("Best C:", grid.best_params_)
print("Best CV ROC AUC:", grid.best_score_)
print("Held-out test accuracy:", log_reg.score(X_test, y_test))

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'C': 0.075, 'class_weight': 'balanced', 'penalty': 'l2', 'solver':
'liblinear'}
Best CV ROC AUC: 0.8535356626237899
Held-out test accuracy: 0.7694915254237288
True Negatives: 142
False Positives: 30
False Negatives: 38
True Positives: 85
FPR = 0.1744 FNR = 0.3089

```

```

[91]: # Shrey - KNN
param_grid = {
    'n_neighbors': [12, 13, 14],
    'weights': ['uniform', 'distance'],
    'p': [1]
}

knn_grid = GridSearchCV(
    neighbors.KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='roc_auc'
)

knn_grid.fit(X_train, y_train)    # already scaled, already DataFrame

best_knn = knn_grid.best_estimator_

print("Best C:", knn_grid.best_params_)
print("Best CV ROC AUC:", knn_grid.best_score_)

```

```

print("Held-out test accuracy:", knn_grid.score(X_test, y_test))

y_pred = best_knn.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'n_neighbors': 12, 'p': 1, 'weights': 'uniform'}
Best CV ROC AUC: 0.8615454081821892
Held-out test accuracy: 0.8542730194743808
True Negatives: 162
False Positives: 10
False Negatives: 43
True Positives: 80
FPR = 0.0581 FNR = 0.3496

```

[92]: *# Val - Random Forest*

```

param_grid = {
    'n_estimators': [250, 300, 340, 380, 400],
    'max_depth': [None, 25, 30],
    'min_samples_split': [10, 15, 20],
    'min_samples_leaf': [3, 4, 6],
    'class_weight': [None, 'balanced'],
}

RF_grid = GridSearchCV(
    ensemble.RandomForestClassifier(random_state=100),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
)

RF_grid.fit(X_train, y_train)

```

```

best_rf = RF_grid.best_estimator_

print("Best C:", RF_grid.best_params_)
print("Best CV ROC AUC:", RF_grid.best_score_)
print("Held-out test accuracy:", RF_grid.score(X_test, y_test))

y_pred = best_rf.predict(X_test)
y_truth = y_test

tn, fp, fn, tp = confusion_matrix(y_truth, y_pred).ravel()

print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

output = ordered_pair(tn, fp, fn, tp)
FPR_axis.append(output[0])
FNR_axis.append(output[1])
print(f'FPR = {output[0]:.04f} FNR = {output[1]:.04f}')

```

```

Best C: {'class_weight': 'balanced', 'max_depth': None, 'min_samples_leaf': 4,
'min_samples_split': 10, 'n_estimators': 300}
Best CV ROC AUC: 0.8748749304551018
Held-out test accuracy: 0.8647428625449045
True Negatives: 150
False Positives: 22
False Negatives: 30
True Positives: 93
FPR = 0.1279 FNR = 0.2439

```

```

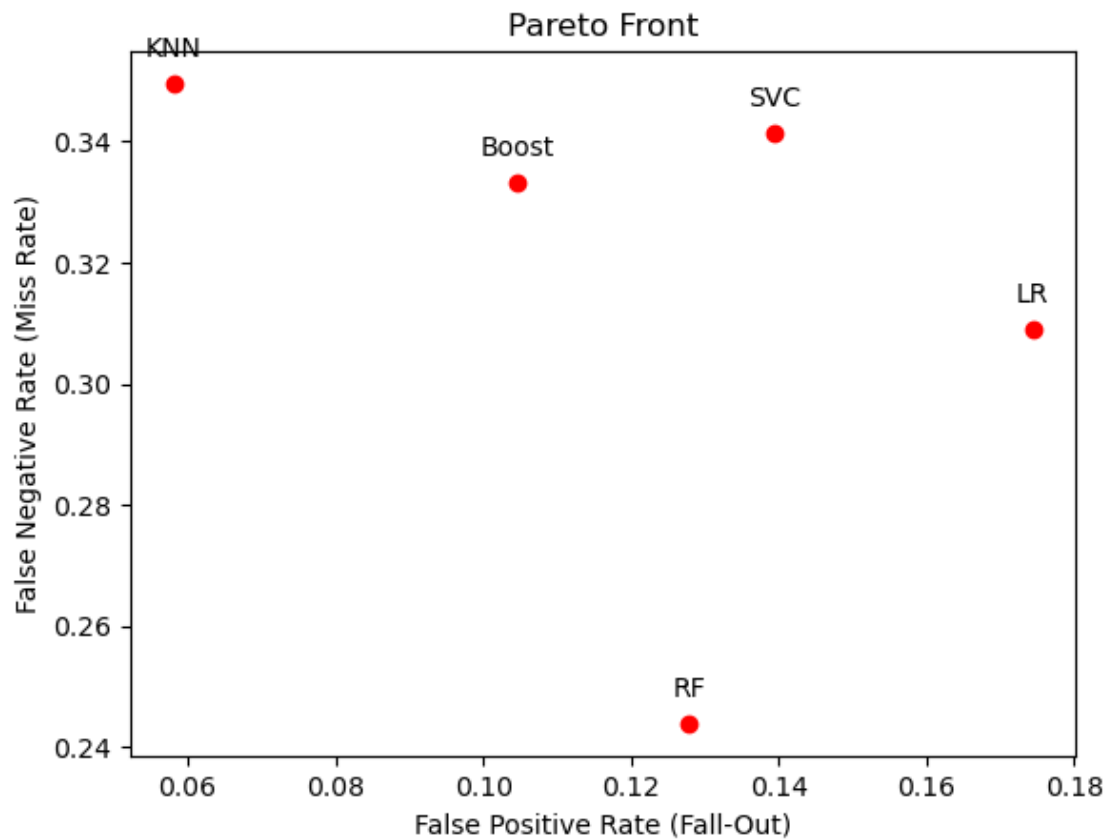
[93]: # Plot output
plt.scatter(FPR_axis, FNR_axis, color='r')

for i, label in enumerate(name_labels):
    # xy=(x, y) is the point position
    # xytext=(offset_x, offset_y) nudges the text so it doesn't overlap the dot
    plt.annotate(
        label,
        (FPR_axis[i], FNR_axis[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center'
    )

plt.xlabel("False Positive Rate (Fall-Out)")

```

```
plt.ylabel("False Negative Rate (Miss Rate)")
plt.title("Pareto Front")
plt.show()
```



6 Final Pareto Frontier

```
[94]: points = np.column_stack([FPR_axis, FNR_axis])    # shape (n, 2)

def pareto_layer(pts):
    """Return indices (into pts) of the non-dominated points."""
    order = np.lexsort((pts[:, 1], pts[:, 0]))    # sort by FPR, then FNR
    sorted_pts = pts[order]
    keep = []
    best_fnr = np.inf
    for idx, (_, y) in zip(order, sorted_pts):
        if y < best_fnr:
            keep.append(idx)
            best_fnr = y
    return np.array(keep)
```

```

# --- Layer 1 (primary front) ---
layer1_idx = pareto_layer(points)
layer1 = points[layer1_idx]

# --- Layer 2 (secondary front) ---
remaining_mask = np.ones(len(points), dtype=bool)
remaining_mask[layer1_idx] = False
layer2_idx = pareto_layer(points[remaining_mask])
# map back to original indices
original_remaining = np.where(remaining_mask)[0]
layer2_idx = original_remaining[layer2_idx]
layer2 = points[layer2_idx]

layer1_full = np.vstack([[0, 1], layer1, [1, 0]])
layer2_full = np.vstack([[0, 1], layer2, [1, 0]])

```

```

[95]: plt.figure(figsize=(8, 6))

plt.scatter(points[:, 0], points[:, 1])

for i, label in enumerate(name_labels):
    plt.annotate(label,
                  (points[i, 0], points[i, 1]),
                  textcoords="offset points",
                  xytext=(0, 10),
                  ha='center')

plt.plot(layer1_full[:, 0], layer1_full[:, 1],
         color='k', linestyle='-', marker='o',
         drawstyle='steps-post', label='Pareto front (layer 1)')

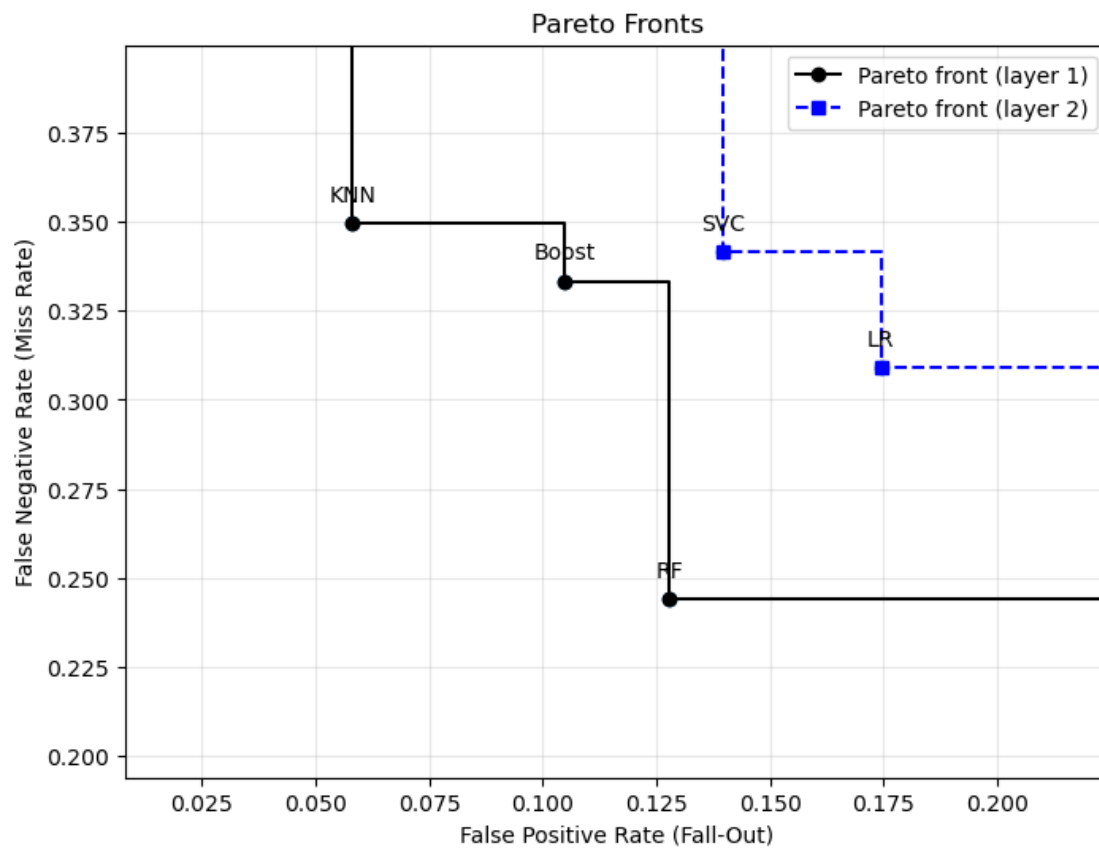
if len(layer2) > 1:
    layer2_full = np.vstack([[0, 1], layer2, [1, 0]])
    plt.plot(layer2_full[:, 0], layer2_full[:, 1],
             color='b', linestyle='--', marker='s',
             drawstyle='steps-post', label='Pareto front (layer 2)')

# Tight window around the actual points - the spanned corners get clipped
pad = .05
plt.xlim(points[:, 0].min() - pad, points[:, 0].max() + pad)
plt.ylim(points[:, 1].min() - pad, points[:, 1].max() + pad)

plt.xlabel("False Positive Rate (Fall-Out)")
plt.ylabel("False Negative Rate (Miss Rate)")
plt.title("Pareto Fronts")
plt.legend()

```

```
plt.grid(alpha=0.3)
plt.show()
```



[]:

[]:

[]: